

MAE 5110: Legged Robots, Assignment 2

Sidharth Rao · smr353

How to Run

(The following scripts pull from configuration of grid resolution set in `assignment_2_config.py`)

Setting up for a specified grid resolution

uv sync

1. Estimate the balancing controller's region of attraction.

uv run python -m assignment_2.assignment_2_roa

2. Generate the Poincaré return map at the upright position (theta=0)

uv run python -m assignment_2.assignment_2_poincare

3. Build a policy to minimize number of steps to balancing based on return map.

uv run python -m assignment_2.assignment_2_step_policy

4. Simulate the policy and generate the main walker animation.

uv run python -m assignment_2.assignment_2

These commands save their results under `assignment_2/output/`:

- `roa/roa_map.png` - balancing-controller region of attraction
- `poincare/poincare_return_maps.png` - Poincaré return maps
- `poincare/poincare_steps.png` - minimum-step policy
- `walker.gif` - simulation animation

Auxiliary Analysis Scripts

The following are additional analyses not necessary for running `assignment_2.py`:

Find and animate the shortest and longest converging paths for an initial state.

uv run python -m assignment_2.assignment_2_longest_path

Plot simulated step counts over an initial angle and velocity sweep.

uv run python -m assignment_2.assignment_2_policy_steps

Compare alpha/velocity grid resolutions for step map to find best resolution.

uv run python -m assignment_2.assignment_2_grid_search

These auxiliary scripts save their images and animations at:

- `assignment_2_longest_path.py`
 - `assignment_2/output/poincare/shortest_converging_path.gif`
 - `assignment_2/output/poincare/longest_converging_path.gif`
- `assignment_2_policy_steps.py`
 - `assignment_2/output/policy_steps/policy_step_counts.png`
- `assignment_2_grid_search.py`
 - `assignment_2/output/grid_search/images/success_rates.png`
 - `assignment_2/output/grid_search/images/success_regions_<alpha samples>x<velocity samples>.png`

Sketches

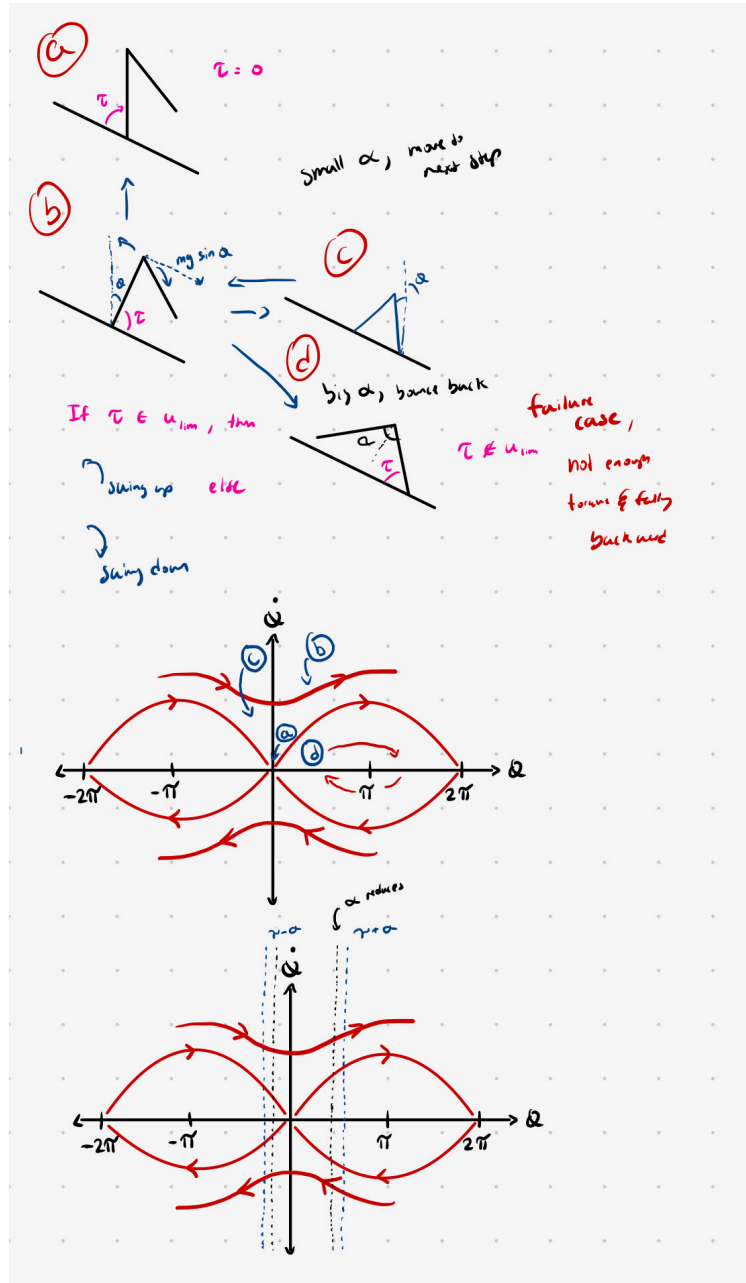


Figure 1: Sketches

Region of Attraction Visualization

(generated via `assignment_2_roa.py`)

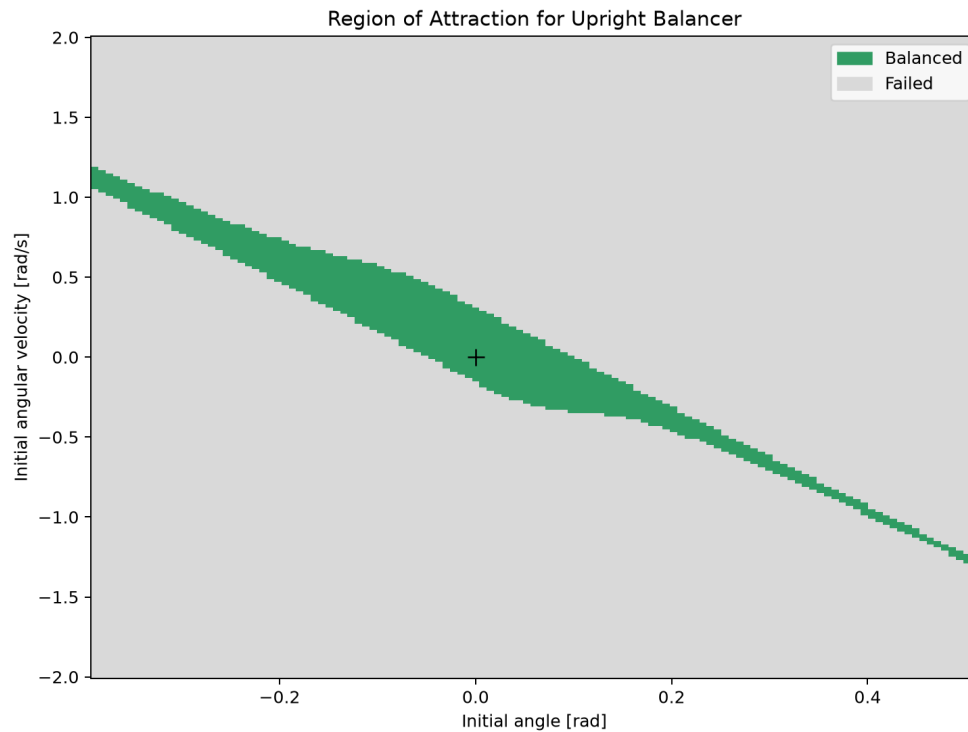


Figure 2: Region of Attraction Map for the Ankle Balancing Controller

Looking at the Region of Attraction map, we can see that there is a thicker low velocity region near the upright angle where the controller is able to come to a balance. As the angle goes away from 0, the velocity needs to have higher and higher magnitude towards the center to have enough energy to balance. Having too much energy causes it to overshoot and not balance because the controller bounds are higher in the negative direction, and so the controller can add more energy in the negative angle direction, but the slope adds energy in the positive angle direction, and so it is easier to balance when moving forwards than backwards.

Poincaré Section Selection

I selected 0 rad (vertical) for the Poincaré section. 0 is definitely transverse to all of the different pendulum paths in the state graph (except to 0 rad with 0 rad/s vel). This choice of Poincaré section also avoids numerical abnormalities that show up at collisions, and is relatively easy to implement. Additionally, the entire alpha range comfortably allows for this section.

(generated via `assignment_2_poincare.py`)

(must set `POINCARÉ_ALPHA_SAMPLES=31` and `POINCARÉ_VELOCITY_SAMPLES=210` in `assignment_2_config.py` to generate this plot)

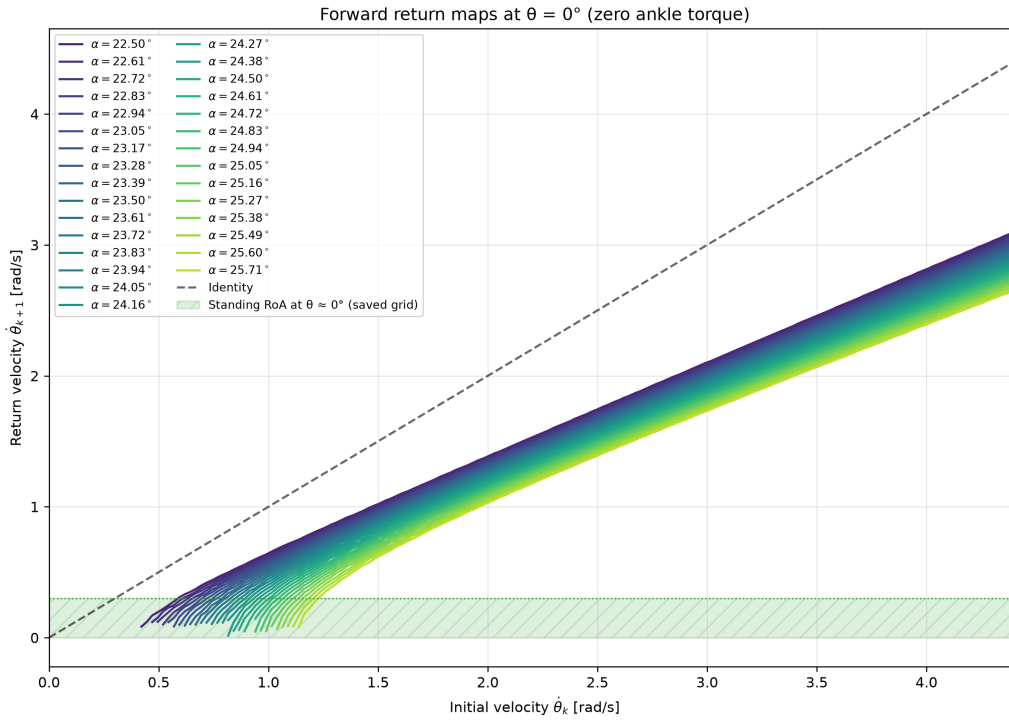


Figure 3: Poincaré Angular Velocity Return Map at Upright Position

Grid Resolution Selection

I initially made a return map - step map that calculated the minimum number of steps needed to reach balancing from different starting velocities at upright. Looking at the below graph, it's clear that there are velocity boundaries where additional steps are needed to reach convergence. There was also a band where nothing converged which was initially very confusing, but I thought maybe there were some initial conditions that didn't have enough energy to enter the RoA of balancing at upright (marked in green).

(generated via `assignment_2_step_policy.py`)

(must set `POINCARÉ_ALPHA_SAMPLES=31` and `POINCARÉ_VELOCITY_SAMPLES=210` in `assignment_2_config.py`, and completely disable the balancing controller in `assignment_2_simulation.py` to generate this plot)

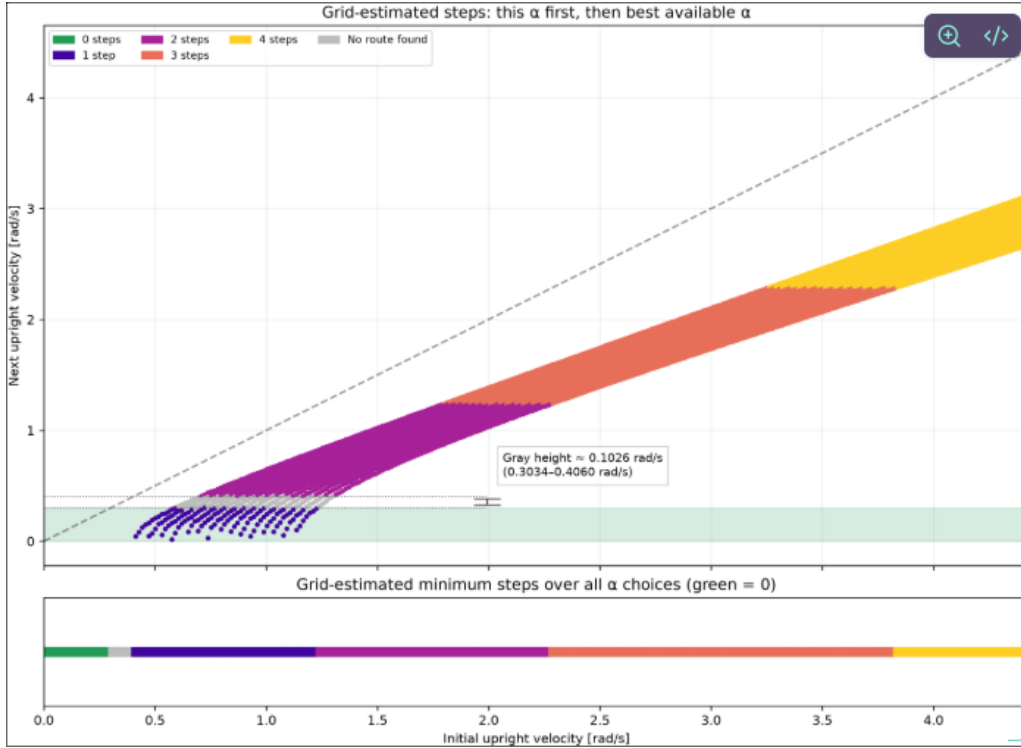


Figure 4: Broken Minimum-step Policy, High-resolution Grid

I tried reducing and greatly increasing the resolution, but was not able to see the grey area completely go away or see any other artifacts appear.

(generated via `assignment_2_step_policy.py`)

(must set `POINCARÉ_ALPHA_SAMPLES=31` in `assignment_2_config.py` and completely disable the balancing controller in `assignment_2_simulation.py` to generate this plot)

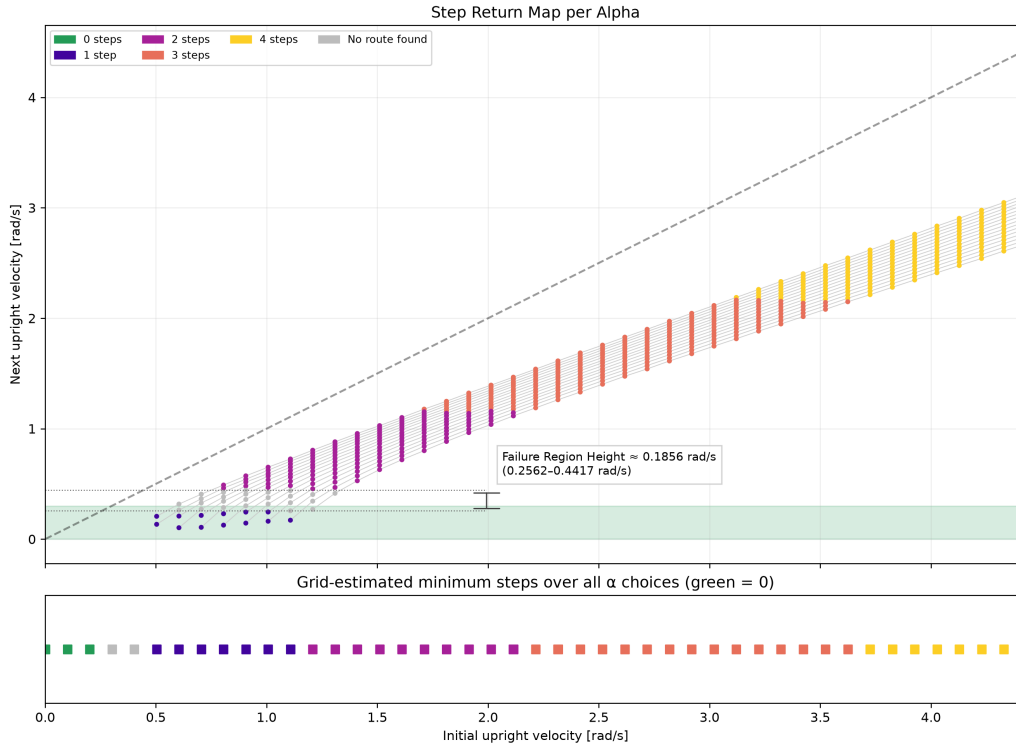


Figure 5: Broken Minimum-step Policy, Low-resolution Grid

I then realized that this was because I wasn't actually turning on the ankle controller whenever the wheel entered the RoA. After fixing this:

(generated via `assignment_2_step_policy.py`)

(must set `POINCARÉ_ALPHA_SAMPLES=31` in `assignment_2_config.py`)

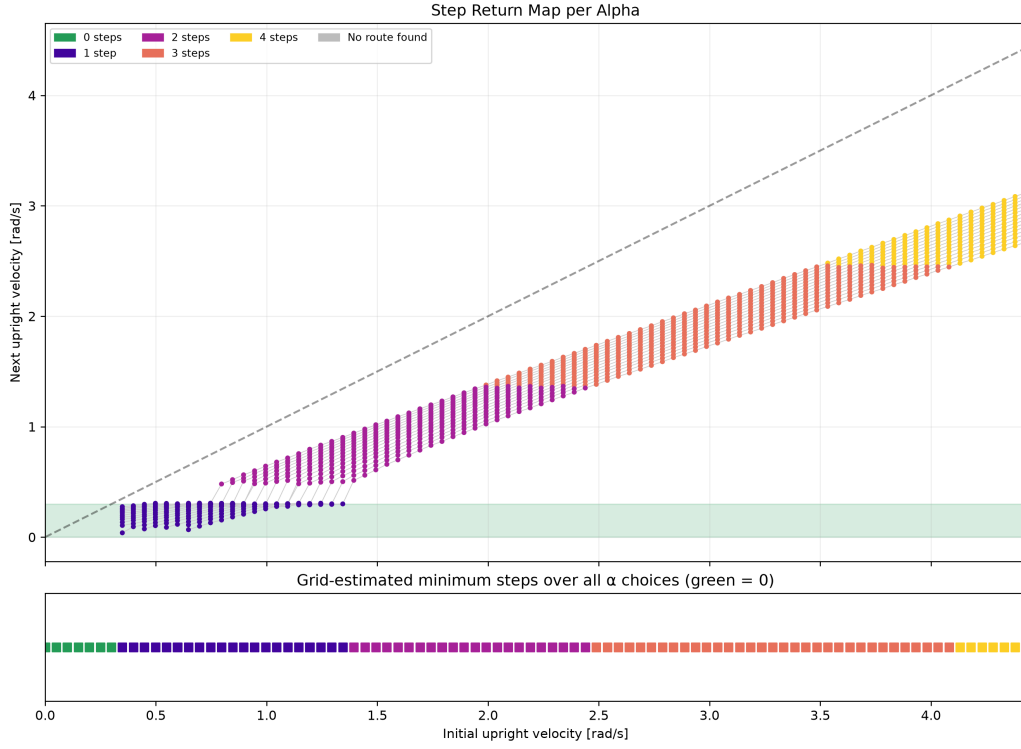


Figure 6: Fixed Minimum-step Policy, Active Controller

Then, I optimized the resolution. I did 4000 random trials per resolution to hone in on the most sparse grid that was still getting 100% balancing rate.

Looking at some results (generated with `assignment_2_grid_search.py`):

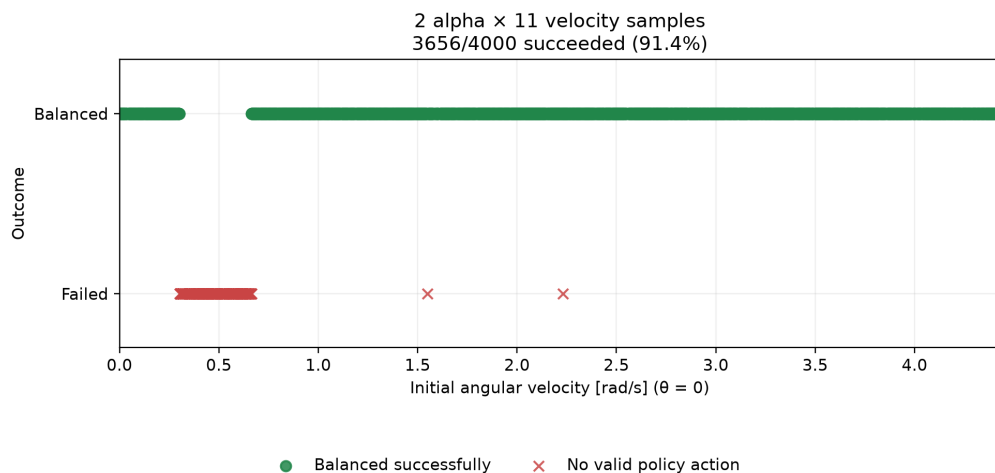


Figure 7: Grid-search outcomes for 2 step-angle samples and 11 velocity samples

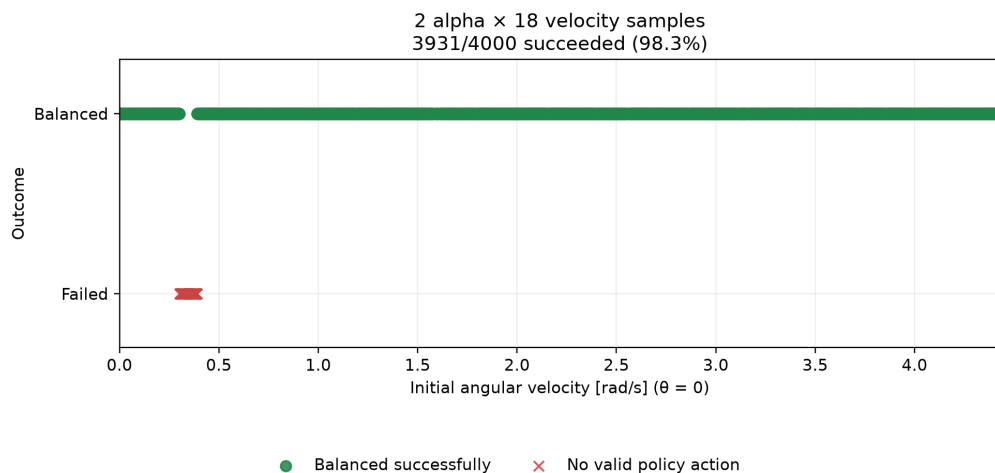


Figure 8: Grid-search outcomes for 2 step-angle samples and 18 velocity samples

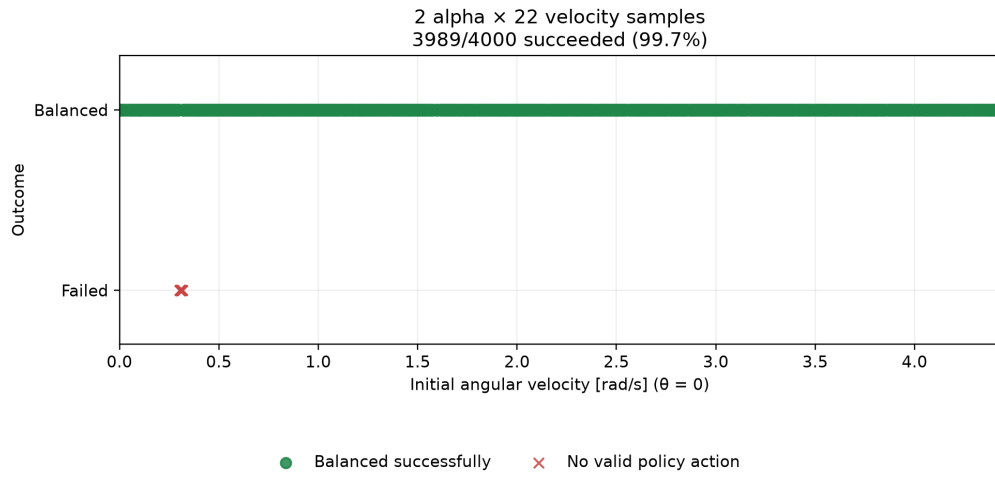


Figure 9: Grid-search outcomes for 2 step-angle samples and 22 velocity samples

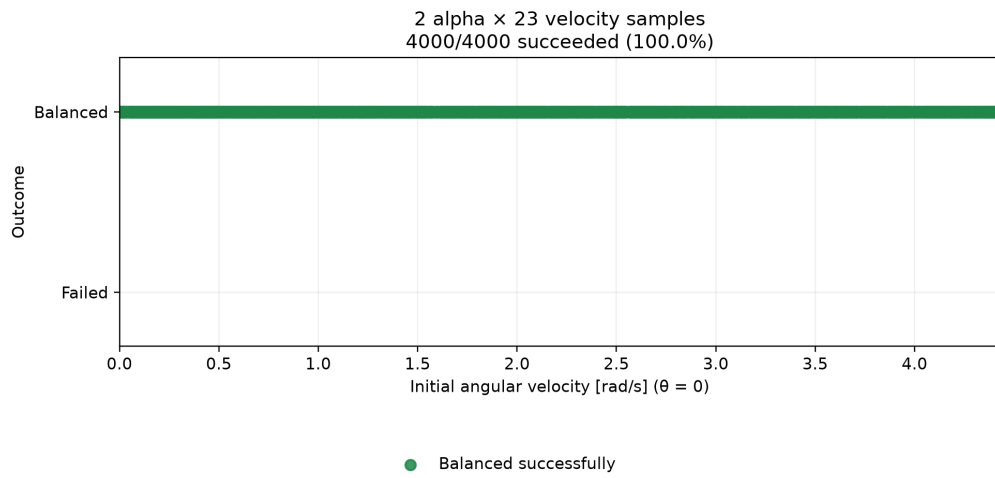


Figure 10: Grid-search outcomes for the selected 2-by-23 resolution policy grid

So 2x23 was the lowest resolution that has a 100% success rate. This case gives this return step map:

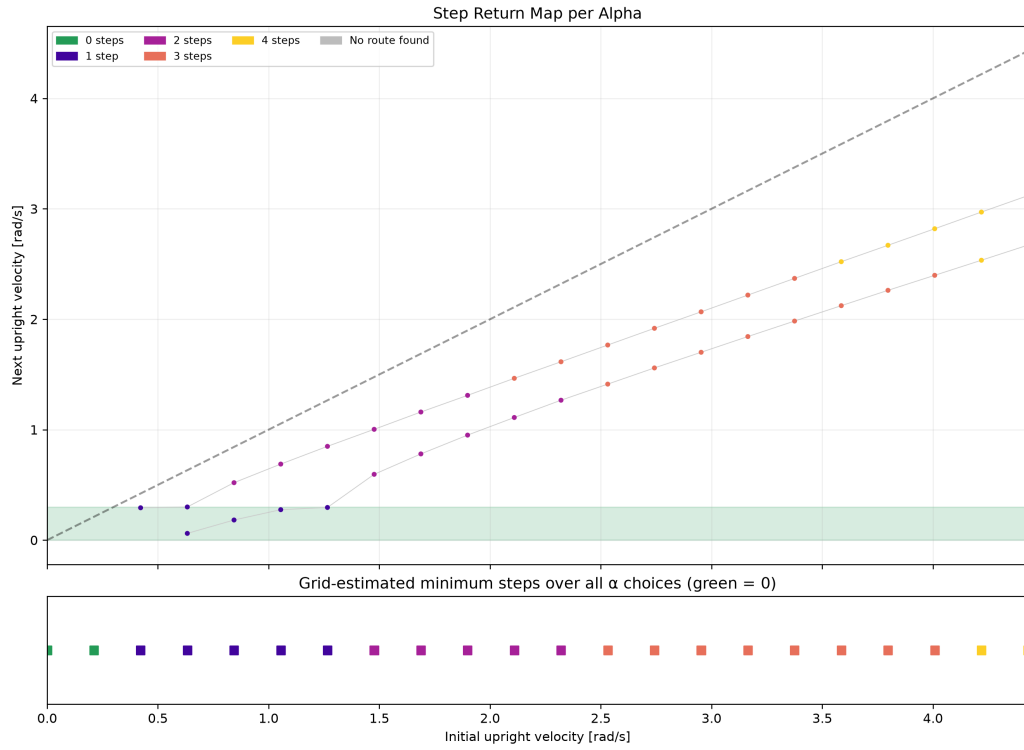


Figure 11: Minimum-step policy for the selected 2-by-23 grid

Trajectory with 3 Steps - Longest Path

The following results were generated with `assignment_2_longest_path.py` .

I selected the upright state $[\theta, \dot{\theta}] = [0 \text{ rad}, 3.0 \text{ rad/s}]$. From this state, the shortest policy path takes three steps to enter the standing controller's region of attraction.

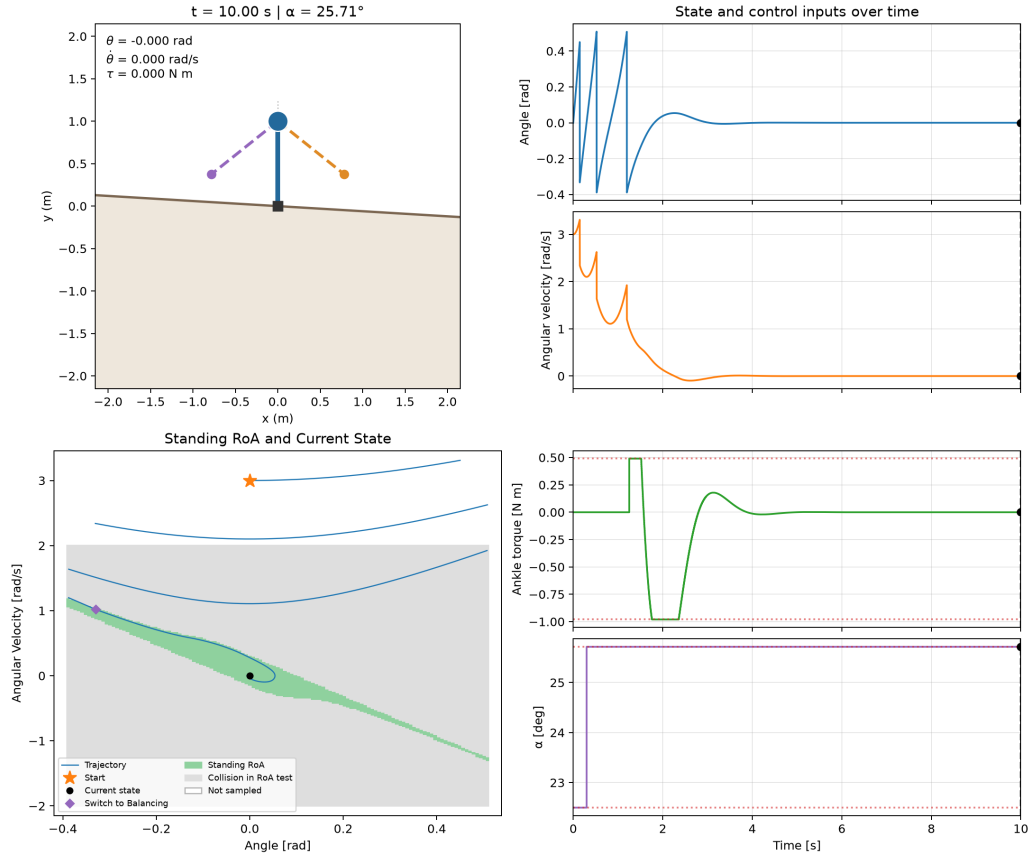


Figure 12: Shortest converging trajectory from the selected initial state: three steps

Then, I performed DFS over the alpha map to figure out the longest path for this starting state that balanced.

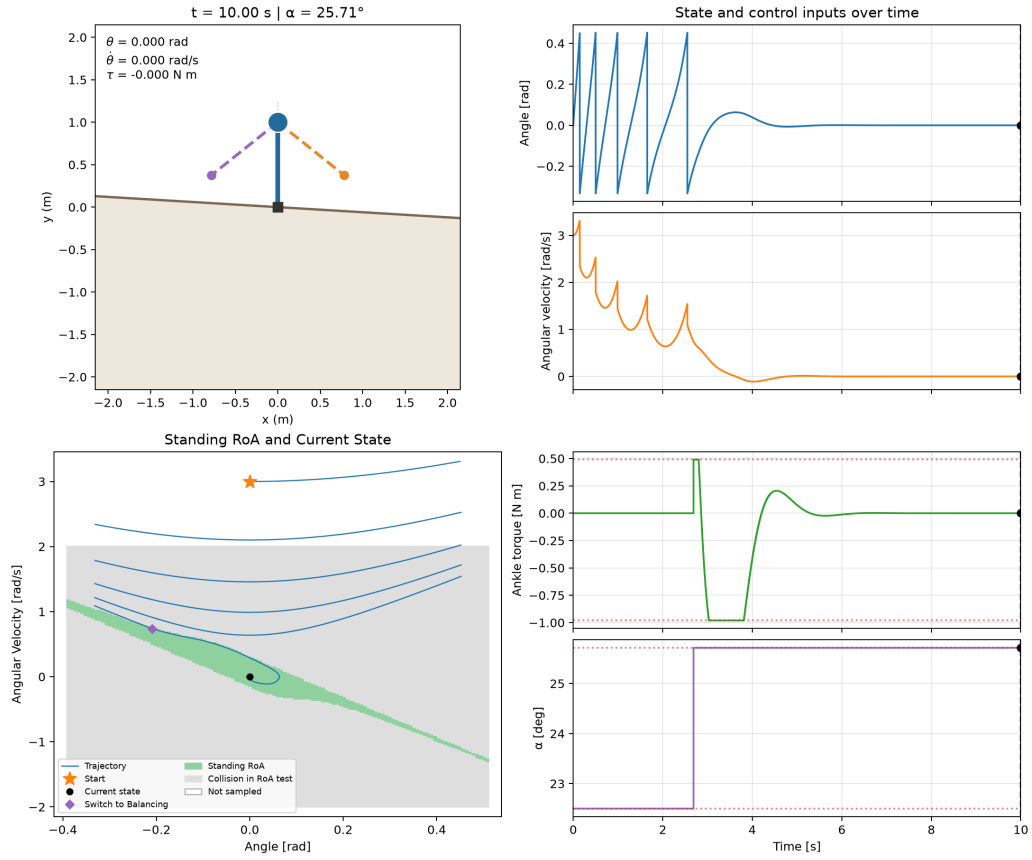


Figure 13: Longest converging trajectory from the selected initial state: five steps

Steps Visualization

This plot visualizes the number of steps it takes to balance / fail from many starting conditions covering the velocity space from 0 to a Froude number of 2 and angular space covering all possible starting angles given maximum alpha.

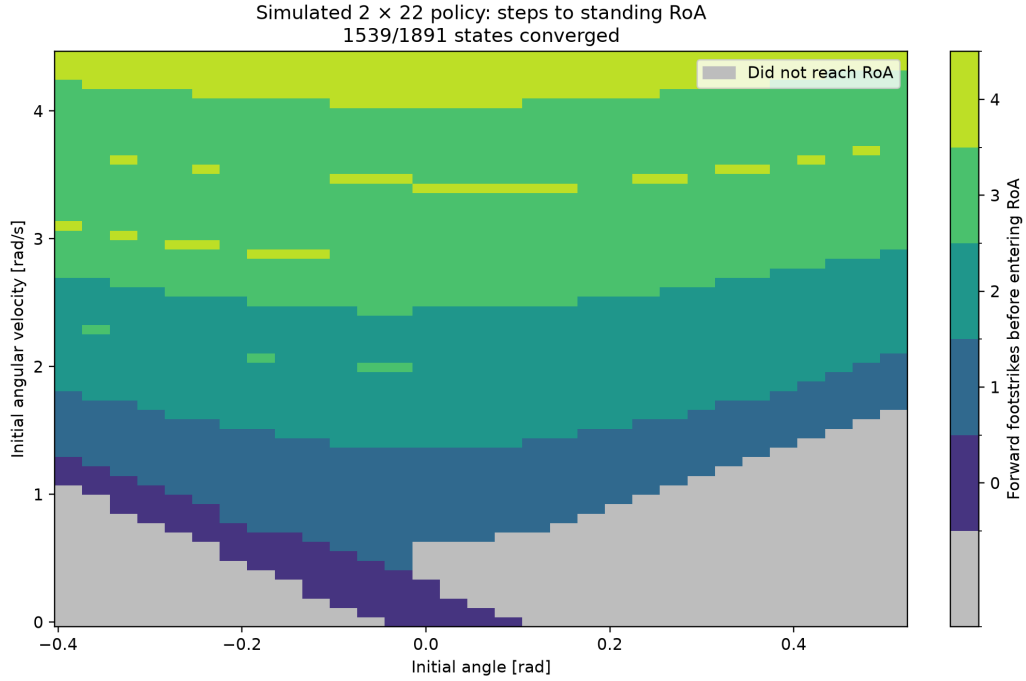


Figure 14: Steps to balance across the sampled initial state space